QA ALM Traceability Test Design

1. Traceability: The Foundation of Confidence

Traceability is critical for **Release Confidence** and **UAT (User Acceptance Testing) Readiness** because it provides mathematical proof of coverage.

- **Impact Analysis:** If a requirement changes last minute, traceability tells you exactly which pieces of code and which test cases are affected. This prevents "regression surprises."
- **Gap Identification:** It allows leads to see if a feature exists but has zero tests assigned to it. You cannot be confident in a release if 20% of your features haven't been validated.
- **UAT Readiness:** Before a user sits down for UAT, traceability ensures the environment is actually ready. It proves that all high-priority "Business Requirements" have passed internal QA, saving the users' time and avoiding embarrassing "broken on arrival" scenarios.

---

2. Risks of Unlinked Requirements and Test Cases

When requirements are "orphaned" (not linked to test cases), the development lifecycle breaks down in several ways:

- **Silent Failures:** A developer might build a feature perfectly, but if there is no linked test case, a future update could break that feature without any alarm bells going off.
- **Scope Creep & Gold Plating:** Without links, it’s hard to tell if a test case is testing a required feature or something "extra" that wasn't asked for, wasting resources.
- **Audit & Compliance Failures:** In regulated industries (like Finance or Healthcare), "unlinked" requirements are a major compliance violation. You must be able to prove *why* and *how* every requirement was tested.
- **Inefficient Debugging:** When a test fails, if it isn't linked to a requirement, the QA team might struggle to understand the "business intent." They won't know if the bug is a minor UI glitch or a total failure of a core business rule.

---

3. Validating AI-Generated Artifacts via Human Quality Gates

AI is excellent at **acceleration**, but it lacks **context** and **accountability**. Human Quality Gates act as the "sanity check" for AI-produced work (like generated code, test scripts, or UI layouts).

The Validation Workflow:

1. **Requirement Alignment (The Context Gate):** AI can generate a functional website, but a human must verify if it matches the *brand voice* and *specific user intent* that an LLM might miss.
2. **Logic & Security Review (The Technical Gate):** AI-generated code can include "hallucinations" or outdated libraries with security vulnerabilities. Human senior developers must perform a peer review to ensure the code is maintainable and secure.
3. **Edge Case Verification (The Logic Gate):** AI tends to focus on the "Happy Path." Human testers are needed to brainstorm the weird, destructive ways a real human might use the software—scenarios an AI wouldn't predict.
4. **Final Ethical & Accessibility Check:** Humans must ensure the AI-generated UI meets **WCAG (Accessibility)** standards and doesn't contain biased language or imagery.

Key Takeaway: AI should be the "Draftsman," but the Human remains the "Architect" and "Sign-off Authority."